

Python Programming – Day 5

Class Notes & Summary

Opening Activity – Student Speaking Session

The instructor opened with a confidence-building speaking activity before class content began. One student was asked to come up and speak on any topic.

The student spoke about a recent trip to Goa with friends — visited Paradise Beach and other beaches, did open-sea water activities, stayed in tents, attended a teacher's wedding, enjoyed a hostel with indoor games, volleyball, and a campfire.

The instructor engaged interactively — asking questions like 'Can you name the beaches?', 'Where did you stay?', 'Did you make new friends?', drawing out more detail and keeping it conversational.

 **Speaking Activity:** *This is a great format — the instructor asked follow-up questions, which transforms a monologue into a real conversation. This is exactly how English communication works in a professional setting (interviews, meetings, standups). Tip to improve further: after one student speaks, ask the rest of the class one question about what they heard. This builds active listening alongside speaking confidence.*

1. Recap – Day 4 Homework Review

The instructor briefly recapped the Day 4 assignment before moving forward:

- Task 1 (Arithmetic): $a=17$, $b=5$ — all 7 operators. Modulus $17\%5 = 2$ (because $5 \times 3 = 15$, remainder is 2). Power $17^{**}5 = 1,419,857$.
- Task 2 (Odd/Even): $\text{number} \% 2 == 0$ means even. 0 is also an even number — it is divisible by 2 with remainder 0.
- The login and grade tasks were verified. Students who completed were moved forward.

 **Instructor Note:** *The instructor mentioned an important interview tip: in an if/elif/else chain, once a condition is true and its block executes, Python does NOT check any remaining elif or else. This 'first match wins and exits' behaviour is a common MCQ trap in technical tests.*

2. if / elif / else – Execution Flow Deep Dive

The 'only one block runs' rule – reinforced

The instructor asked a class exercise: if there are 10 lines — odd lines are if/elif/else conditions, even lines are print statements — and the first if is false, third is true, fifth and seventh are true, ninth is else — which print statements execute?

Answer: Only line 4 (under the first true condition, line 3). Because as soon as line 3 is true and line 4 executes, Python exits the entire if/elif/else block. It does not check lines 5, 7, or 9.

💡 **Instructor Note:** *This is a genuine interview question pattern. Interviewers often write a chain of if/elif conditions where multiple conditions would be true if checked, and ask which lines execute. The answer is always: only the FIRST matching block. After that, execution jumps past the entire chain.*

Short-circuit evaluation with 'and'

Demonstrated with the login example from Day 4:

```
if username == 'admin' and password == '123':
```

- If username is wrong → this condition is False. Python does NOT check the password at all because of 'and'. Both must be true — if the first is false, the result is already false.
- If username is correct but password is wrong → the username check is True, but password check is False → combined is False → moves to elif.

💡 **Instructor Note:** *The short-circuit is not just a Python quirk — it is how 'and' works in logic. If the first operand of AND is false, the whole expression is false regardless of the second. Interviewers test this. Knowing this also helps you order your conditions efficiently: put the cheapest or most likely to fail check first.*

3. Mini Project – ATM Simulator (Homework Bonus, Done in Class)

The ATM simulator was built live to consolidate all concepts from Days 3-5. Here is the logic flow:

```
actual_pin = 1234
balance = 5000
pin = int(input('Enter PIN: '))

if pin == actual_pin:
    amount = int(input('Enter withdrawal amount: '))
    if amount <= 0:
        print('Invalid amount')
```

```

elif amount > balance:
    print('Insufficient balance')
else:
    balance = balance - amount
    print('Withdrawal successful. New balance:', balance)
else:
    print('Incorrect PIN')

```

Two separate if/else blocks are nested — one for PIN, one for amount. Once the PIN block is true, it enters the amount block. These are independent chains.

 **Instructor Note:** *Nested if/else is very common in real applications. Every time you have a condition that only makes sense after a previous check passes, you nest. In the ATM: checking the balance only makes sense AFTER the PIN is verified. This is intentional, logical structuring — not just syntax.*

4. for Loop

What is a loop?

A loop lets you repeat a block of code multiple times without writing it again and again. Instead of writing `print('hello')` 100 times, you write a loop.

for loop with range()

```

for i in range(100):
    print(i)

```

range(n)	Runs from 0 to n-1. <code>range(100)</code> → 0, 1, 2 ... 99
range(start, n)	Runs from start to n-1. <code>range(50, 100)</code> → 50 to 99
range(start, n, step)	Runs from start to n-1, jumping by step. <code>range(50, 100, 2)</code> → 50, 52, 54...

 **Instructor Note:** *`range(100)` runs 100 times (0 through 99). If you want the number 100 to appear, write `range(101)`. The ending number is always excluded — this is Python's design. In other languages, off-by-one errors with loop boundaries are a very common bug. Always verify your start and end values.*

Iterating directly over a list

Python lets you loop through a list without using index numbers — something most other languages cannot do as cleanly:

```
grades = ['A', 'B', 'C', 'D', 'A', 'B']
for g in grades:
    print(g)
```

In each iteration, the variable `g` takes the next value from the list. It acts like a pointer moving through the items one by one.

5. while Loop

A while loop runs as long as a condition is True. Unlike for loop (which has a fixed range), while loop is condition-driven.

```
i = 100
while i >= 0:
    print(i)
    i -= 1    # same as i = i - 1
```

for loop	Use when you know HOW MANY times to repeat (fixed range or iterating a list)
while loop	Use when you repeat UNTIL A CONDITION changes (unknown number of iterations)

Infinite loop warning

If you forget to update the variable inside the while loop, the condition never becomes False and the loop runs forever:

```
i = 100
while i >= 0:
    print(i)
    # forgot i -= 1 → infinite loop!
```

 **Instructor Note:** The instructor demonstrated a real infinite loop live — the system kept printing forever until manually stopped (Ctrl+C). Always ask yourself: 'Is there a point where this condition will become False?' If not, you have an infinite loop. The while loop is heavily used in operating systems and semaphores — it keeps a process running until a condition (like a lock being free) changes.

6. break and continue

break – stop the loop immediately

```
for i in range(10):
    if i == 5:
        break
    print(i)
# Output: 0 1 2 3 4
```

When break is hit, the loop terminates completely at that point. Even if range is 1,000,000, it stops at 5.

continue – skip this iteration, go to next

```
for i in range(10):
    if i == 5:
        continue
    print(i)
# Output: 0 1 2 3 4 6 7 8 9 (5 is skipped)
```

When continue is hit, whatever code remains in the loop body for that iteration is skipped. The loop then immediately moves to the next iteration.

break	Exits the loop entirely. No more iterations.
continue	Skips the rest of THIS iteration. Loop continues with the next value.

💡 **Instructor Note:** A good way to remember: *break = stop the whole loop. continue = skip this one, keep going. Common use: continue to skip invalid data while processing a list. break to stop searching once you find what you need.*

7. Mini Game – Number Guessing (random module)

The instructor built a quick number-guessing game to apply everything learned so far:

```
import random
secret = random.randint(1, 10)
guess = int(input('Guess a number between 1 and 10: '))
if guess == secret:
    print('You won!')
else:
    print('Wrong! The number was:', secret)
```

`random.randint(1, 10)` picks a random integer between 1 and 10 (both inclusive). This is a built-in Python module — no installation needed, just import it.

 **Instructor Note:** This game brings together 5 concepts at once: `import`, `random` module, `int(input())`, `if/else`, and comparison operators. Building small games is one of the best ways to solidify programming concepts — the instructor used this deliberately as a revision tool. The instructor noted the number kept coming up as 2 — because `random.randint` generates a new number each run, and with a small range (1-10), repetition is natural.

8. Python Data Structures – Introduction

Python has 4 built-in data structures. Each has a specific purpose:

list	Ordered, mutable (changeable), duplicates allowed. Use <code>[]</code>
tuple	Ordered, immutable (cannot change), duplicates allowed. Use <code>()</code>
set	Unordered, no duplicates, cannot access by index. Use <code>{ }</code>
dictionary	Key-value pairs, ordered (Python 3.7+), no duplicate keys. Use <code>{ }</code> ← next class

9. List – Methods & Operations

A list stores multiple items in order. Unlike most languages, Python lists can mix data types.

```
nums = [30, 20, 10]    # basic list
```

Adding items

<code>nums.append(40)</code>	Adds 40 at the END of the list. → <code>[30, 20, 10, 40]</code>
<code>nums.insert(1, 15)</code>	Inserts 15 at index 1. → <code>[30, 15, 20, 10]</code>
<code>nums.extend([50, 60])</code>	Adds multiple items from another list to the end.

Removing items

<code>nums.remove(20)</code>	Removes first occurrence of value 20 (by VALUE, not index).
-------------------------------------	---

<code>nums.pop()</code>	Removes and returns the LAST item.
<code>nums.pop(1)</code>	Removes item at index 1.

Other useful methods

<code>nums.sort()</code>	Sorts ascending (smallest to largest).
<code>nums.sort(reverse=True)</code>	Sorts descending (largest to smallest).
<code>nums.reverse()</code>	Reverses the current order of the list.
<code>nums.count(10)</code>	Returns how many times 10 appears in the list.
<code>nums.index(30)</code>	Returns the index position of value 30.
<code>nums.clear()</code>	Removes all items. List becomes empty [].

 **Instructor Note:** The instructor made an important point: you don't need to memorise every method. What matters is knowing **WHAT** you can do with a list — add, remove, sort, search, count. When you need the exact syntax, look it up. Knowing the capability exists is what matters. This is how professional developers work.

10. Tuple

A tuple is like a list but immutable — once created, it cannot be changed (no adding, removing, or modifying items).

```
t = (30, 10, 20)    # parentheses instead of square brackets
```

- You CANNOT append, insert, remove, or sort a tuple. Trying to do so gives: `AttributeError: 'tuple' object has no attribute 'append'.`
- You CAN do: `t.count(10)`, `t.index(30)`, `max(t)`, `min(t)`.
- `type(t)` returns `<class 'tuple'>`.

When to use tuple vs list:

list	Data that will change — items will be added, removed, or updated.
tuple	Data that should stay fixed — config values, coordinates, database rows.

 **Instructor Note:** The instructor mentioned: in Django (the Python web framework), tuples are used extensively — particularly for things like URL patterns, database field choices, and settings. The reason is safety: you don't want those values accidentally modified at runtime. Knowing when to use tuple vs list shows professional thinking.

11. Set

A set stores unique items only. Duplicates are automatically removed. The order is not guaranteed (unordered).

```
s = {10, 30, 10, 40, 30}    # curly braces
print(s)    # Output: {10, 30, 40} ← duplicates gone, order may vary
```

Key properties

- No duplicates — if you add the same item twice, it only stores once.
- Unordered — you cannot access items by index (`s[0]` gives an error).
- You cannot predict the print order — it changes between runs.

Set methods

s.add(5)	Adds a single item to the set.
s.remove(10)	Removes 10. Raises <code>KeyError</code> if 10 not in set.
s.discard(10)	Removes 10. NO error if 10 not in set (safer version).
s.pop()	Removes a RANDOM item (since set is unordered).

Set operations (from school maths)

Given: $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$

A B (Union)	All unique items from both sets: $\{1, 2, 3, 4, 5\}$
A & B (Intersection)	Only items in BOTH sets: $\{3\}$
A - B (Difference)	Items in A that are NOT in B: $\{1, 2\}$

 **Instructor Note:** A brilliant real-world use of sets: removing duplicates from a list. If you have a list `[30, 30, 30, 40, 10]` and want only unique values, wrap it in `set()`: `list(set([30, 30, 30, 40, 10])) → [30, 40, 10]`. This is a one-line deduplication trick used constantly in data processing.

 **Instructor Note:** The instructor also told students to push all class code to GitHub. This is excellent professional advice — every assignment you write and store on GitHub builds your public portfolio. Employers check GitHub. Even if the code is simple today, having a consistent commit history shows discipline and progression.

12. Coming Up – Dictionary (Next Class)

Dictionary is the fourth built-in data structure. It stores data as key-value pairs — similar to JSON. This is what `response.json()` returns when making API calls. It will be covered in full in the next class.

13. Quick Reference – Day 5

for i in range(n)	Loop n times. i goes 0 to n-1.
for item in list	Loop through each item in a list directly.
while condition:	Loop as long as condition is True.
break	Stop the loop immediately.
continue	Skip current iteration, go to next.
i -= 1	Same as <code>i = i - 1</code> . Used to decrement in while loops.
random.randint(a, b)	Pick a random integer between a and b (inclusive).
list.append(x)	Add x to end of list.
list.insert(i, x)	Insert x at index i.
list.remove(x)	Remove first occurrence of value x.
list.pop()	Remove and return last item.
list.sort()	Sort ascending.
list.sort(reverse=True)	Sort descending.
tuple = (a, b, c)	Immutable ordered collection. Cannot be changed.
set = {a, b, c}	Unordered, no duplicates.
set.add(x)	Add item to set.
set.discard(x)	Remove safely (no error if missing).
A B	Set union (all unique items from both).
A & B	Set intersection (items in both).
A - B	Set difference (in A but not in B).

End of Day 5 Notes